

Scheduling

Choosing which ready process or thread runs next

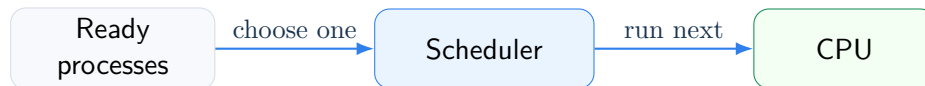
Definition

Scheduling is the operating system activity of choosing which ready process or thread should run on the CPU next.

1. Basic Idea

Why Scheduling Is Needed

In a multiprogrammed system, several processes or threads may be ready to run at the same time. If there is only one CPU, the operating system must choose one of them.



Term	Meaning
Scheduler	The part of the operating system that chooses the next process or thread to run.
Scheduling algorithm	The rule or strategy used by the scheduler to make that choice.
Ready state	The process or thread can run, but is waiting for CPU time.

2. Processes and Threads

Same Basic Problem

Many scheduling ideas apply to both processes and threads: the CPU is limited, and several runnable execution units may compete for it.

Kernel Threads

When the kernel manages threads, scheduling is usually done per thread. The scheduler may care little about which process owns each thread.

3. Historical Motivation

Batch Systems

Early batch systems often ran jobs one after another. With multiprogramming, multiple jobs and users could wait for service, so scheduling became more important.

Batch and Timesharing Together

A mainframe may need to choose between a background batch job and an interactive terminal user. The scheduler affects both system efficiency and user satisfaction.

4. Why Scheduling Matters More in Some Systems

System Type	Scheduling Importance
Simple personal computer	Often only one process is actively demanding the CPU, so scheduling may not be very visible.
CPU-heavy workstation	Rendering, compiling, simulation, and other heavy tasks can make CPU scheduling important.
Network server	Many processes may compete to serve users, so choosing the right one matters.
Mobile or sensor device	CPU, memory, and battery may be limited, so scheduling may also consider energy use.

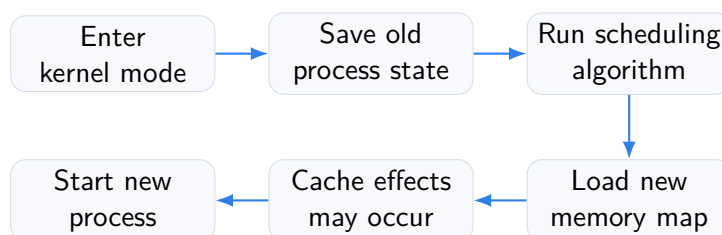
Server Example

If a server must choose between collecting daily statistics and serving a user request, users usually benefit if the request-handling process runs first.

5. Cost of Process Switching

Switching Is Not Free

Changing from one process to another takes CPU time. Too many switches per second can waste a significant amount of processing power.



Typical Work During a Switch

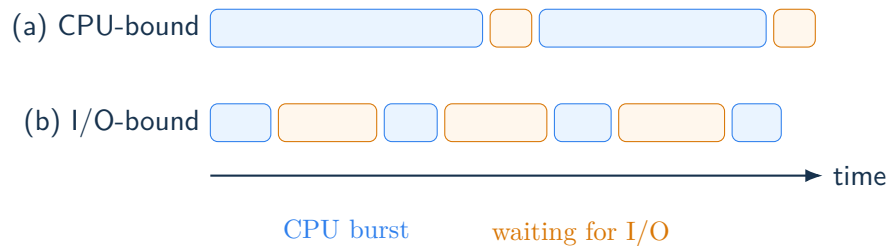
- switch from user mode to kernel mode,
- save registers and other state of the current process,
- choose a new ready process,
- reload memory-management information for the new process,
- restart execution in the selected process,
- possibly reload cache and related hardware state.

6. Process Behavior

CPU Burst

A **CPU burst** is a period during which a process is actively using the CPU before it blocks, usually for I/O.

Most processes alternate between CPU bursts and waiting for I/O such as disk or network operations.



7. CPU-Bound and I/O-Bound Processes

Type	CPU Burst Pattern	Meaning
CPU-bound	Long CPU bursts, infrequent I/O waits.	The process spends most of its time computing.
I/O-bound	Short CPU bursts, frequent I/O waits.	The process often blocks waiting for external devices.

Key Factor

A process is I/O-bound because it does little computation between I/O requests. The important scheduling clue is the length of the CPU burst, not the length of the I/O operation.

Why I/O-Bound Processes Often Deserve Quick Service

If an I/O-bound process gets the CPU quickly, it can issue its next disk or network request quickly. This helps keep I/O devices busy and improves overall system balance.

8. When to Schedule

Scheduling Decision Point

A **scheduling decision point** is a moment when the operating system may need to choose a process or thread to run.

Situation	Why Scheduling May Be Needed
New process is created	Both parent and child may be ready, so the scheduler chooses which runs next.
Process exits	The exiting process cannot continue, so another ready process must be chosen.
Process blocks	A process waiting for I/O, a semaphore, or another event cannot use the CPU.
I/O interrupt occurs	A blocked process may become ready after the device completes its work.
Clock interrupt occurs	A running process may have used up its allowed CPU time.

Idle Process

If no ordinary process is ready, the system normally runs an idle process until useful work becomes available.

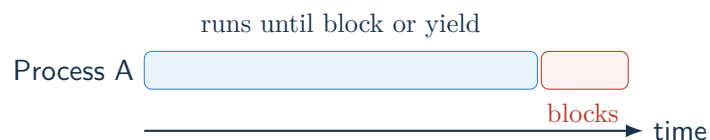
9. Nonpreemptive Scheduling

Nonpreemptive Scheduling

A **nonpreemptive** scheduling algorithm lets the chosen process run until it blocks or voluntarily gives up the CPU.

Main Property

Clock interrupts do not normally force the current process off the CPU. After interrupt handling, the same process usually resumes.



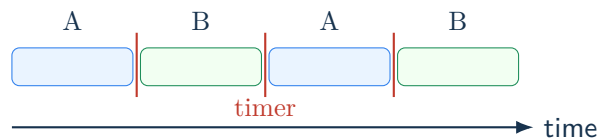
10. Preemptive Scheduling

Preemptive Scheduling

A **preemptive** scheduling algorithm lets a process run for at most a fixed time interval. If it is still running when the interval ends, it can be suspended.

Clock Requirement

Preemptive scheduling requires a clock interrupt or timer mechanism so control returns to the scheduler.



11. Categories of Scheduling Algorithms

Different Environments, Different Goals

Scheduling algorithms are designed differently because batch systems, interactive systems, and real-time systems optimize for different outcomes.

Category	Description
Batch	Jobs run without an impatient user waiting at a terminal. Long time slices or nonpreemptive algorithms may be acceptable.
Interactive	Users expect quick responses. Preemption is important so one process cannot monopolize the CPU.
Real time	Programs have deadlines or timing constraints. The scheduler must make timing behavior reliable.

12. Batch Scheduling

Batch Environment

Batch systems run jobs such as payroll, inventory, billing, banking calculations, or insurance processing. Users usually wait for final output, not immediate interaction.

Why Long Runs Can Be Acceptable

Because there is no interactive user waiting for each keystroke response, batch systems can often use nonpreemptive scheduling or long time periods. This reduces process-switching overhead.

13. Interactive Scheduling

Interactive Environment

Interactive systems include personal computers, terminals, and many servers. They must keep service responsive for users.

Need for Preemption

Without preemption, one CPU-hungry or buggy process could keep running and deny service to other users or programs.

Foreground Before Background

If a user opens a file while a background task is checking email, the user-visible request should normally get quick CPU service.

14. Real-Time Scheduling

Real-Time Environment

Real-time systems run tasks with timing requirements. A process may have to run before a deadline to avoid losing data or degrading output quality.

Difference from General Interactive Systems

Real-time systems usually run programs designed for the application. Interactive systems may run arbitrary programs, including badly behaved ones.

15. Goals for All Systems

Goal	Meaning
Fairness	Comparable processes should receive comparable service.
Policy enforcement	The scheduler should enforce stated system priorities and rules.
Balance	The CPU, memory, and I/O devices should be kept busy when possible.

Balance Example

A good mix of CPU-bound and I/O-bound processes can keep both the CPU and disk busy. Running all CPU-bound jobs first and all I/O-bound jobs later can leave one resource idle while another is overloaded.

16. Batch System Goals

Goal	Meaning
Throughput	Maximize the number of jobs completed per hour.
Turnaround time	Minimize the time from job submission to job completion.
CPU utilization	Keep the CPU busy as much as possible.

Throughput vs. Turnaround

A scheduler that always runs short jobs may complete many jobs per hour, but long jobs may wait too long or even starve if short jobs keep arriving.

CPU Utilization Caution

High CPU utilization is useful as a capacity signal, but it is not the whole goal. Users usually care more about completed work and waiting time.

17. Interactive System Goals

Goal	Meaning
Response time	Minimize the time between a user request and the system's response.
Proportionality	Make response times match user expectations about how long tasks should take.

Proportionality

Users expect uploading a large video to take time, but they expect disconnecting from the server afterward to be quick. A short-looking operation that takes too long feels worse than a long-looking operation that takes time.

18. Real-Time System Goals

Goal	Meaning
Meeting deadlines	Run required tasks before their timing deadlines.
Predictability	Avoid irregular timing that causes quality degradation, especially in multimedia.

Multimedia Example

Occasional delay may not crash the system, but irregular audio scheduling can quickly reduce perceived quality. Predictable timing matters.

19. Scheduling Goals Summary

Environment	Main Goals
All systems	Fairness, policy enforcement, balance.
Batch	Throughput, turnaround time, CPU utilization.
Interactive	Response time, proportionality.
Real time	Meeting deadlines, predictability.

20. Simple Scheduler Sketch

```
while system is running do
  if current process blocks, exits, or is preempted then
    choose a ready process according to the scheduling policy
    switch to the chosen process
  end if
end while
```

Policy Matters

The scheduler's policy determines what best means: shortest waiting time, highest priority, earliest deadline, best response time, or some other goal.

21. Big Picture

Main Conclusion

Scheduling decides which ready process or thread gets the CPU next. Good scheduling depends on the environment: batch systems care about completed jobs, interactive systems care about quick response, and real-time systems care about timing guarantees.

Exam Shortcut

Remember: scheduler chooses the next ready process; CPU-bound means long CPU bursts; I/O-bound means short CPU bursts; nonpreemptive runs until block or yield; preemptive uses timer interrupts; different systems have different scheduling goals.

22. Quick Review

Key Points

- Scheduling is needed when multiple ready processes or threads compete for one CPU.
- The scheduler is the OS component that chooses what runs next.
- The scheduling algorithm is the rule used to make that choice.
- Process switching has overhead, so excessive switching wastes CPU time.
- Processes alternate CPU bursts with I/O waits.
- CPU-bound processes have long CPU bursts.
- I/O-bound processes have short CPU bursts and frequent I/O waits.
- Scheduling decisions happen on creation, exit, blocking, I/O interrupts, and clock interrupts.
- Nonpreemptive scheduling lets a process run until it blocks or yields.
- Preemptive scheduling can suspend a process after a time interval.
- Batch, interactive, and real-time systems have different scheduling goals.
- Fairness, policy enforcement, and balance matter in all systems.
- Batch systems emphasize throughput, turnaround time, and CPU utilization.
- Interactive systems emphasize response time and proportionality.
- Real-time systems emphasize deadlines and predictability.